

Practical 2 - REST API with Flask

Python file extension: app.py (.py)

Step 1: Check Python

```
python --version
```

Step 2: Create/Open Project Folder

```
cd path\to\Flask_Project cd
C:\Users\Nikhita\Documents\Flask_Project
```

Step 3: Create and Activate Virtual Environment

```
python -m venv venv
venv\Scripts\activate
```

Step 4: Install and Verify Flask

```
pip install flask pip
show flask python -m
flask --version
```

Step 5: Create app.py

```
from flask import Flask
app = Flask(__name__)
@app.route('/') def
home():
    return "Hello, Flask!" if
__name__ == '__main__':
app.run(debug=True)
```

Step 6: Run Flask

```
python app.py
```

Open: <http://127.0.0.1:5000>

Step 7: Replace app.py with REST API Code

```
from flask import Flask, jsonify, request
app = Flask(__name__) # Sample data
storage todos = [
    {
        "id": 1,
        "title": "Learn Flask",
        "completed": False
    }
]
# CREATE a new task
@app.route('/todos', methods=['POST'])
def create_todo():
    data = request.get_json()
    new_todo = {
        "id": len(todos) + 1,
        "title": data['title'],
        "completed": False
    }
    todos.append(new_todo)
    return jsonify(new_todo), 201
# READ all tasks
@app.route('/todos', methods=['GET'])
def get_todos(): return
jsonify(todos) # READ a single task
@app.route('/todos/<int:id>', methods=['GET'])
def get_todo(id):
    todo = next((t for t in todos if t['id'] == id), None)
    if todo is None: return jsonify({"message": "Task not
found"}), 404 return jsonify(todo)
# UPDATE a task
@app.route('/todos/<int:id>', methods=['PUT'])
def update_todo(id):
```

```

    todo = next((t for t in todos if t['id'] == id), None)
    if todo is None:
        return jsonify({"message": "Task not found"}), 404
    data = request.get_json()
    todo['title'] = data.get('title', todo['title'])
    todo['completed'] = data.get('completed', todo['completed'])
    return jsonify(todo)
# DELETE a task
@app.route('/todos/<int:id>', methods=['DELETE'])
def delete_todo(id):
    global todos
    todo = next((t for t in todos if t['id'] == id), None)
    if todo is None:
        return jsonify({"message": "Task not found"}), 404
    todos = [t for t in todos if t['id'] != id]
    return jsonify({"message": "Task deleted successfully"})
if __name__ == '__main__':
    app.run(debug=True)

```

Step 8: Run and Test CRUD with curl

```

python app.py
curl http://127.0.0.1:5000/todos -X POST -H "Content-Type: application/json" -d '{"title": "Complete REST API Assignment"}'
curl -X PUT http://127.0.0.1:5000/todos/1 -H "Content-Type: application/json" -d '{"title": "Learn Flask API", "completed": true}'
curl -X DELETE http://127.0.0.1:5000/todos/1
curl http://127.0.0.1:5000/todos

```

Practical 3A - Git Repository Management

Python file extension: hello.py (.py)

Step 1: Open VS Code and Terminal

Step 2: Go to Desktop and Create Project

```
cd Desktop mkdir
Git_Practical cd
Git_Practical code
.
```

Step 3: Create hello.py

```
print("Hello Git!")
```

Step 4: Create README.md

```
# Git Practical This is my first
Git repository.
Project Name: Python Demo
Created by: Your Name
```

Step 5: Initialize Git git

```
init
```

Step 6: Check Status and Stage Files

```
git status
git add .
git status
```

Step 7: Configure Git (First Time Only)

```
git config --global user.name "Your Name" git
config --global user.email "your@email.com"
```

Step 8: Commit Project

```
git commit -m "Initial Python Project"
git status git log
```

Step 9: Modify hello.py

```
print("Hello Git!") print("Welcome
to Git Repository")
```

Step 10: Stage and Commit Updated File

```
git status git add hello.py git
commit -m "Updated hello.py" git
log
```

Practical 3B - Branching, Pull Request and Merge

Python file extension: hello.py (.py)

Step 1: Open Git_Practical in VS Code and Terminal

Step 2: Check/Rename Branch

```
git branch git
branch -M main git
branch
```

Step 3: Create Feature Branch

```
git checkout -b feature-login
git branch
```

Step 4: Modify hello.py

```
print("Hello Git!") print("Welcome
to Git Repository") print("Login
Feature Added") print("Feature
Branch Example")
```

Step 5: Stage and Commit

```
git status
git add hello.py
git commit -m "Added login feature"
git log --oneline
```

Step 6: Create GitHub Repository

Repository name: Git_Practical

Step 7: Connect Remote

```
git remote add origin https://github.com/yourusername/Git_Practical.git
git remote -v
```

Step 8: Push Main and Feature Branch

```
git checkout main
git push -u origin main
git checkout feature-login
git push -u origin feature-login
```

Step 9: Create Pull Request on GitHub

Compare & Pull Request -> Create Pull Request

Step 10: Merge Pull Request

Merge Pull Request -> Confirm Merge

Step 11: Pull Updated Main

```
git checkout main
git pull origin main
```

Step 12: Verify / Optional Delete Branch

```
git branch
git branch -d feature-login
git push origin --delete feature-login
```

Practical 3C - Simulate and Resolve Merge Conflict

Python file extension: hello.py (.py)

Step 1: Open Project and Check Branch

```
git branch
```

Step 2: Create Conflict Branch

```
git checkout -b feature-conflict
```

Step 3: Change hello.py in Feature Branch

```
print("Hello Git!")
print("Feature Branch Version")
```

Step 4: Commit Feature Change

```
git status
git add hello.py
git commit -m "Modified hello.py in feature branch"
```

Step 5: Switch to Main

```
git checkout main
```

Step 6: Change Same Line in Main

```
print("Hello Git!")
print("Main Branch Version")
```

Step 7: Commit Main Change

```
git add hello.py
git commit -m "Modified hello.py in main branch"
```

Step 8: Merge Feature Branch

```
git merge feature-conflict
```

Step 9: Resolve hello.py - Keep Both

```
print("Hello Git!") print("Main  
Branch Version") print("Feature  
Branch Version")
```

Step 10: Stage Resolved File

```
git add hello.py
```

Step 11: Complete Merge

```
git commit -m "Resolved merge conflict"
```

Step 12: Verify

```
git log --oneline  
git status
```

Practical 4 - Virtual Machine and Flask Deployment

Python file extension: app.py (.py)

Step 1: Open VMware Workstation

Power on the Ubuntu Virtual Machine and log in.

Step 2: Update Ubuntu

```
sudo apt update  
sudo apt upgrade -y
```

Step 3: Verify/Install Python and Flask

```
python3 --version  
sudo apt install  
python3 python3-pip -y  
pip3 install  
flask pip3 show flask
```

Step 4: Check VM IP Address

```
ip addr  
hostname -I
```

Step 5: Set VMware Network Mode

VM -> Settings -> Network Adapter -> NAT (or Bridged if required)

Step 6: Verify Network and Ports

```
ping google.com  
sudo ss -tln  
netstat -tln
```

Step 7: Allow Flask Port

```
sudo ufw allow 5000  
sudo ufw status
```

Step 8: Create Flask Project

```
mkdir FlaskApp  
cd FlaskApp
```

Step 9: Create app.py

```
from flask import Flask  
app = Flask(__name__)  
@app.route('/')  
def home():  
    return "Hello from Ubuntu Virtual Machine!"  
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=5000)
```

Step 10: Run Application

```
python3 app.py
```

Step 11: Test Application

```
http://localhost:5000  
http://<VM_IP_Address>:5000
```

Step 12: Stop Server

Ctrl + C

Practical 5A - Dockerize a Flask Application

Python file extension: app.py (.py)

Step 1: Install/Open Docker Desktop

Open Docker Desktop and wait for Docker Engine to show Running.

Step 2: Verify Docker

```
docker --version
docker info
```

Step 3: Create Project Folder

```
mkdir Flask_Project
cd Flask_Project
code .
```

Step 4: Create app.py

```
from flask import Flask
app = Flask(__name__)
@app.route("/") def
home():
    return "Welcome to Dockerized Flask Application"
@app.route("/about") def about():
    return "Docker Practical" if
__name__=="__main__":
app.run(host="0.0.0.0",port=5000)
```

Step 5: Create Virtual Environment and Install Flask

```
python -m venv venv
venv\Scripts\activate
pip install flask
```

Step 6: Create requirements.txt

```
Flask
```

Step 7: Create Dockerfile

```
FROM python:3.12-slim
WORKDIR /app COPY
requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Step 8: Build Docker Image

```
docker build -t flask-app .
```

Step 9: Verify Image

```
docker images
```

Practical 5B - Run and Test Docker Container

Step 1: Open Docker Desktop

Wait until status shows Engine Running.

Step 2: Open Project Folder

```
cd C:\Users\YourName\Documents\Flask_Project
```

Step 3: Verify/Build Image

```
docker images docker build
-t flask-app .
```

Step 4: Run Container

```
docker run -d -p 5000:5000 flask-app
```

Step 5: Verify Container

```
docker ps
```

Step 6: Test in Browser

```
http://localhost:5000  
http://localhost:5000/about
```

Step 7: Test with curl

```
curl http://localhost:5000
```

Step 8: Check Logs

```
docker logs <container_id>
```

Step 9: Stop Container

```
docker ps | grep <container_id>  
docker stop <container_id>
```

Step 10: Restart Container

```
docker ps -a | grep <container_id>  
docker start <container_id>  
http://localhost:5000
```

Practical 5C - Push Docker Image to Docker Hub

Step 1: Check Image

```
docker images
```

Step 2: Log in to Docker Hub

```
docker login
```

Step 3: Check Logged-in Account

```
docker info
```

Step 4: Tag Image

```
docker tag flask-app nikhita123/flask-app:latest
```

Step 5: Verify Tag

```
docker images
```

Step 6: Push Image

```
docker push nikhita123/flask-app:latest
```

Step 7: Verify on Docker Hub

```
Open Docker Hub -> Repositories -> flask-app
```

Practical 6 - Docker Compose Multi-Container Application

Python file extension: app.py (.py)

Step 1: Create app.py

```
from flask import Flask
import psycopg2
import os
app = Flask(__name__)
@app.route('/')
def home():
    try:
        conn = psycopg2.connect(
            host=os.getenv("DB_HOST"),
            database=os.getenv("DB_NAME"),
            user=os.getenv("DB_USER"),
            password=os.getenv("DB_PASSWORD")
        )
        conn.close()
        return "Connected Successfully to PostgreSQL!"
    except Exception as e:
        return "Database Connection Failed: " + str(e)
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 2: Create requirements.txt

```
Flask
psycopg2-binary
```

Step 3: Create Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Step 4: Create .env

```
DB_HOST=db
DB_NAME=mydatabase
DB_USER=postgres
DB_PASSWORD=postgres
```

Step 5: Create docker-compose.yml

```
version: "3.9"
services:
  web:
    build: .
    container_name: flask_app
    ports:
      - "5000:5000"
    depends_on:
      - db
    environment:
      DB_HOST: db
      DB_NAME: mydatabase
      DB_USER: postgres
      DB_PASSWORD: postgres
  db:
    image: postgres:16
    container_name: postgres_db
    restart: always
    environment:
      POSTGRES_DB: mydatabase
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
volumes:
  postgres_data:
```

Step 6: Build and Start

```
cd FlaskComposeProject
docker compose build
docker compose up -d
```

Step 7: Verify and Test

```
docker ps
http://localhost:5000
docker compose logs web
docker compose logs db
```

Step 8: Stop

```
docker compose down
```

Practical 7 - CI/CD with Jenkins

Python files: app.py and test_app.py (.py)

Step 1: Check Java and Git

```
java -version
git --version
```

Step 2: Install/Open Jenkins

Install Jenkins for Windows -> open <http://localhost:8080>

Step 3: Unlock Jenkins

Open C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword and paste the password. **Step**

4: Install Suggested Plugins

Select Install suggested plugins and create administrator account.

Step 5: Create Project Files

Create folder: flask-jenkins-demo

Step 6: Create app.py

```
from flask import Flask
app = Flask(__name__)
@app.route("/") def
home():
    return "Hello from Flask CI/CD!"
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 7: Create requirements.txt

```
Flask
pytest
```

Step 8: Create test_app.py

```
from app import app
def test_home():
    client = app.test_client() response =
client.get("/") assert response.status_code == 200
assert response.data == b"Hello from Flask CI/CD!"
```

Step 9: Test Locally

```
python -m venv venv
venv\Scripts\activate pip
install -r requirements.txt
pytest python app.py
```

Step 10: Push to GitHub

```
git init
git add .
git commit -m "Initial Flask application" git
branch -M main git remote add origin
YOUR_GITHUB_REPOSITORY_URL git push -u origin
main
```

Step 11: Create Jenkins Pipeline

Jenkins Dashboard -> New Item -> Flask-CI-CD -> Pipeline -> OK

Step 12: Add Pipeline Script

```
pipeline { agent any
stages {
stage('Checkout') {
steps { git branch:
'main',
url: 'YOUR_GITHUB_REPOSITORY_URL'
}
}
}
```

```
    stage('Install Dependencies') {      steps {      bat
'python -m pip install -r requirements.txt'
    }
  }
  stage('Test') {
  steps {      bat
'pytest'
    }
  }
}
```

Step 13: Run Pipeline

Save -> Build Now -> open Console Output

Practical 8 - Prometheus and Grafana Monitoring on Windows

Python file extension: app.py (.py)

Step 1: Download and Extract Prometheus

Extract Prometheus to C:\prometheus

Step 2: Configure C:\prometheus\prometheus.yml

```
global:
  scrape_interval: 5s
scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]
  - job_name: "python-app"
    static_configs:
      - targets: ["localhost:8000"]
  - job_name: "windows"
    static_configs:
      - targets: ["localhost:9182"]
```

Step 3: Start Prometheus

```
cd C:\prometheus
prometheus.exe
```

Step 4: Install Python Packages

```
pip install flask prometheus-client
```

Step 5: Create app.py

```
from flask import Flask from prometheus_client import
start_http_server, Counter app = Flask(__name__)
REQUEST_COUNT = Counter("app_requests_total", "Total number of requests")
@app.route("/")
def home():
    REQUEST_COUNT.inc() return "Flask
App Running" if __name__ ==
 "__main__":
    start_http_server(8000)
    app.run(host="0.0.0.0", port=5000)
```

Step 6: Run Flask App

```
python app.py
```

Step 7: Open Prometheus

http://localhost:9090

Step 8: Install/Run Windows Exporter

Install Windows Exporter, then open http://localhost:9182/metrics

Step 9: Install/Open Grafana

Open http://localhost:3000

Step 10: Add Prometheus Data Source

Connections -> Data sources -> Add data source -> Prometheus -> URL: http://localhost:9090 -> Save & test

Step 11: Create Dashboard

Dashboards -> New -> New dashboard -> Add visualization -> select Prometheus

Step 12: Use Queries

```
app_requests_total
windows_cpu_time_total
windows_os_physical_memory_free_bytes
```

Practical 9 - Three-Tier Student Management Application

Python backend file extension: app.py (.py)

Step 1: Check Installed Software

```
python --version
node --version
npm --version
"C:\Program Files\PostgreSQL\18\bin\psql.exe" --version netstat
-ano | findstr :5433
```

Step 2: Create Project

```
cd %USERPROFILE%\Desktop
mkdir three-tier-student-app
cd three-tier-student-app
mkdir backend mkdir frontend
```

Step 3: Create PostgreSQL Database in pgAdmin

```
Servers -> PostgreSQL 18 -> Databases -> Create -> Database
Database: studentdb2
```

Step 4: Create students Table and Data

```
CREATE TABLE students ( id
SERIAL PRIMARY KEY, name
VARCHAR(100) NOT NULL, email
VARCHAR(100) NOT NULL,
course VARCHAR(100) NOT NULL
);
INSERT INTO students (name, email, course)
VALUES
('Rahul', 'rahul@gmail.com', 'BCA'),
('Priya', 'priya@gmail.com', 'BSc CS'),
('Amit', 'amit@gmail.com', 'BCA');
```

Step 5: Create Backend Environment

```
cd %USERPROFILE%\Desktop\three-tier-student-app cd
backend python -m venv venv venv\Scripts\activate python
-m pip install flask flask-cors "psycopg[binary]"
```

Step 6: Create backend/app.py

```
from flask import Flask, jsonify, request
from flask_cors import CORS
import psycopg
app = Flask(__name__)
CORS(app)
# PostgreSQL connection
DB_CONFIG = {
    "host": "localhost",
    "port": 5433,
    "dbname": "studentdb2",
    "user": "postgres",
    "password": "YOUR_POSTGRES_PASSWORD"
}
def get_connection():
    return psycopg.connect(*DB_CONFIG)
@app.route("/") def home():
    return jsonify({
        "message": "Student Management API is running"
    })
@app.route("/students", methods=["GET"])
def get_students():
    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        "SELECT id, name, email, course FROM students ORDER BY id"
    )
    students = cur.fetchall()
    cur.close()
    conn.close()
    result = []
    for student in students:
        result.append({
            "id": student[0],
            "name": student[1],
            "email": student[2],
            "course": student[3]
        })
    return jsonify(result)
@app.route("/students", methods=["POST"])
def add_student():
    data = request.get_json()
    name = data.get("name")
    email =
```

```

data.get("email") course =
data.get("course") conn =
get_connection() cur = conn.cursor()
cur.execute(
    """
    INSERT INTO students (name, email, course)
    VALUES (%s, %s, %s)
    RETURNING id
    """,
    (name, email, course)
)
student_id = cur.fetchone()[0]
conn.commit() cur.close()
conn.close() return jsonify({
    "message": "Student added successfully",
    "id": student_id
}), 201 if __name__ ==
"__main__":
app.run(debug=True, port=5000)

```

Step 7: Run/Test Backend

```

python app.py
http://127.0.0.1:5000/
http://127.0.0.1:5000/students

```

Step 8: Create React Frontend

```

cd %USERPROFILE%\Desktop\three-tier-student-app npm
create vite@latest frontend -- --template react cd
frontend npm install npm install axios code .

```

Step 9: Replace frontend/src/App.jsx

```

import { useEffect, useState } from "react";
import axios from "axios"; import "./App.css";
function App() { const [students,
setStudents] = useState([]); useEffect(() =>
{
  axios
    .get("http://127.0.0.1:5000/students")
    .then((response) => {
      setStudents(response.data);
    })
    .catch((error) => {
      console.error("Error fetching students:", error);
    });
}, []);
return (
  <div className="container">
    <h1>Student Management System</h1>
    <table>
      <thead>
        <tr>
          <th>ID</th>
          <th>Name</th>
          <th>Email</th>
          <th>Course</th>
        </tr>
      </thead>
      <tbody>
        {students.map((student) => (
          <tr key={student.id}>
            <td>{student.id}</td>
            <td>{student.name}</td>
            <td>{student.email}</td>
            <td>{student.course}</td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
);
} export default
App;

```

Step 10: Replace frontend/src/App.css

```

body { font-family: Arial,
sans-serif; background:

```

```
#f2f2f2; margin: 0; padding:
40px;
}
.container { max-
width: 900px; margin:
auto; background:
white; padding: 30px;
border-radius: 10px;
} h1 { text-align:
center;
} table { width: 100%;
border-collapse: collapse;
margin-top: 25px;
} th, td { border: 1px
solid #ccc; padding:
12px; text-align: left;
} th { background:
#eeeeee; }
```

Step 11: Run Frontend

```
npm run dev
http://localhost:5173/
```

Practical 10 - Dockerize Three-Tier Application

Python backend file extension: app.py (.py)

Step 1: Create Project Folders

```
cd %USERPROFILE%\Desktop
mkdir three-tier-docker
cd three-tier-docker
mkdir backend mkdir
frontend mkdir database
```

Step 2: Create backend/app.py

```
from flask import Flask, jsonify
import psycopg2 import os
app = Flask(__name__)
@app.route("/") def
home():
    return "Flask Backend is Running"
@app.route("/students") def
students(): try:
    conn = psycopg2.connect(
host=os.getenv("DB_HOST"),
database=os.getenv("DB_NAME"),
user=os.getenv("DB_USER"),
password=os.getenv("DB_PASSWORD")
)
    cur = conn.cursor() cur.execute("SELECT id, name, email, course FROM
students ORDER BY id") rows = cur.fetchall() cur.close()
conn.close() result = [] for row in rows: result.append({ "id":
row[0],
    "name": row[1],
    "email": row[2],
    "course": row[3]
})
    return jsonify(result)
except Exception as e:
    return jsonify({"error": str(e)}), 500
if __name__ == "__main__":
app.run(host="0.0.0.0", port=5000)
```

Step 3: Create backend/requirements.txt

```
Flask psycopg2-
binary flask-cors
```

Step 4: Create backend/Dockerfile

```
FROM python:3.12-slim
WORKDIR /app COPY requirements.txt . RUN pip
install --no-cache-dir -r requirements.txt COPY .
.
EXPOSE 5000
CMD ["python", "app.py"]
```

Step 5: Create React Frontend

```
cd ..
cd frontend npm create vite@latest . -- --
template react y npm install npm install
axios
```

Step 6: Replace frontend/src/App.jsx

```
import { useEffect, useState } from "react";
import axios from "axios"; function App() {
  const [students, setStudents] = useState([]);
  useEffect(() => {    axios
    .get("http://localhost:5000/students")
    .then((response) => {
      setStudents(response.data);
    })
    .catch((error) => {
      console.error(error);
    });
  }, []);
  return (
    <div>
      <h1>Student Management System</h1>
      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>Course</th>
          </tr>
        </thead>
        <tbody>
          {students.map((student) => (
            <tr key={student.id}>
              <td>{student.id}</td>
              <td>{student.name}</td>
              <td>{student.email}</td>
              <td>{student.course}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
} export default
App;
```

Step 7: Create frontend/Dockerfile

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 5173
CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

Step 8: Create database/init.sql

```
CREATE TABLE IF NOT EXISTS students (
  id SERIAL PRIMARY KEY,  name
  VARCHAR(100) NOT NULL,  email
  VARCHAR(100) NOT NULL,  course
  VARCHAR(100) NOT NULL
); INSERT INTO students (name, email,
course)
VALUES
('Rahul', 'rahul@gmail.com', 'BCA'),
('Priya', 'priya@gmail.com', 'BSc CS'),
('Amit', 'amit@gmail.com', 'BCA');
```

Step 9: Create docker-compose.yml

```
services:
  frontend:
    build: ./frontend
    container_name: student_frontend
```

```

ports:      - "5173:5173"
depends_on:  - backend  backend:
  build: ./backend
  container_name: student_backend
ports:      - "5000:5000"
environment:
  DB_HOST: database
  DB_NAME: studentdb
  DB_USER: postgres
  DB_PASSWORD: postgres
depends_on:  -
database: database:
  image: postgres:16
container_name: student_database
restart: always  environment:
  POSTGRES_DB: studentdb
  POSTGRES_USER: postgres
  POSTGRES_PASSWORD: postgres
ports:      - "5432:5432"
volumes:
  - postgres_data:/var/lib/postgresql/data
  - ./database/init.sql:/docker-entrypoint-initdb.d/init.sqlvolumes:  postgres_data:

```

Step 10: Build and Start

```

docker compose build
docker compose up -d
docker ps docker
compose ps

```

Step 11: Test

```

http://localhost:5000/
http://localhost:5000/students
http://localhost:5173

```

Step 12: View Logs

```

docker compose logs frontend
docker compose logs backend
docker compose logs database

```

Step 13: Stop / Restart

```

docker compose down
docker compose up -d
docker compose ps

```